

Lab 16: On Call — Four Incidents, One Cluster (70 min)

The last lab of the course is a drill. Nothing new is introduced: every incident below is caused by something you learned to prevent on Days 1–3, and every tool you need to find it you have already used. The difference is that this time nobody tells you what is wrong.

Learning Objectives

By the end of this lab you will be able to:

- Work an incident in a fixed order — cluster health, then statement, then range, then waits, then storage — instead of guessing
- Recognise the four signatures that cause most CockroachDB pages: a hot range, a contended row, a placement constraint nothing satisfies, and a job holding garbage collection back
- Choose between a database-side fix (DDL, a job command) and an application-side fix (a deploy), and verify that the fix took
- Restart nodes under load without an outage, and explain why the connection string decides whether that is true

Prerequisites

- The Docker lab cluster (`scripts/crdb`), as for every other lab
- Days 2–3: the DB Console pages (Overview, SQL Activity, Insights, Hot Ranges, Jobs), `SHOW RANGES` , `crdb_internal` , zone configurations, changefeeds

Setup

```
scripts/crdb up
scripts/incident status
```

`scripts/incident` is the breaker. `scripts/incident start N` puts the cluster into incident *N* and prints nothing else; `scripts/incident stop N` puts everything back. Run one incident at a time.

The "application" is real code you can read. Each incident's application runs in a container and executes `/tmp/oncall/N/loop.sh` over and over. Reading that file is the equivalent of reading your service's code; editing it is the equivalent of deploying a fix — the running container picks the change up on its next loop. Some incidents are fixed in the database, some in that file, and part of the exercise is deciding which.

The order of operations

Work every page in this order. Skipping ahead is how on-call engineers spend an hour on the wrong hypothesis.

Step	Question	Where to look
1	Is the cluster healthy?	Console Overview : live nodes, unavailable / under-replicated ranges, CPU per node
2	Is it one statement?	SQL Activity → Statements sorted by latency; Insights
3	Is it one range?	Hot Ranges ; <code>SHOW RANGES ... WITH DETAILS</code>
4	Is something waiting on something?	Transactions (contention column); <code>crdb_internal.transaction_contention_events</code> ; Jobs
5	Is something growing?	Console Storage ; <code>SHOW RANGES</code> range size vs table size; protected timestamps

Keep an incident log as you go — the table under *Deliverables*. Time-to-diagnose is the number that improves with practice; write it down honestly.

Tasks

Part A: Incident 1 — "Ingest is slow and one node is pegged" (12 min)

```
scripts/incident start 1
```

The page: *Telemetry ingest latency is 5× normal. Node 1 is at high CPU; nodes 2 and 3 are nearly idle. Nothing was deployed.*

- Step 1 — cluster health.** Console → Overview. All three nodes live, no unavailable ranges, CPU very uneven. Uneven CPU with a healthy cluster is your first hint that the *work* is uneven, not the cluster.
- Step 2 — one statement?** SQL Activity → Statements, sort by execution count. One `INSERT INTO oncall.events ...` dominates. Its latency is high but its plan is trivial.
- Step 3 — one range?** Console → **Hot Ranges**. One range carries essentially all the writes, and its leaseholder is node 1. Confirm from SQL:

```
SELECT range_id, lease_holder, range_size_mb
FROM [SHOW RANGES FROM TABLE oncall.events WITH DETAILS];
SHOW CREATE TABLE oncall.events;
```

One range, one leaseholder — and a `SERIAL` primary key. Every insert lands at the right edge of the keyspace (Lab 3). The table will eventually split, but each new range is again the rightmost one, so the hotspot moves rather than disappears.

4. **Fix — in the database, online.** The application's `INSERT` does not name `id`, so the key design can change underneath it:

```
ALTER TABLE oncall.events ALTER PRIMARY KEY USING COLUMNS (id) USING HASH WITH
(bucket_count = 16);
```

The primary-key change runs as an online schema change while the inserts continue.

5. **Verify** (give it about a minute):

```
SELECT count(*) AS ranges, count(DISTINCT lease_holder) AS leaseholders
FROM [SHOW RANGES FROM TABLE oncall.events WITH DETAILS];
```

Many ranges, three leaseholders; on Hot Ranges the writes are spread across ~16 ranges at roughly equal rates and the CPU curves on Overview converge.

Debrief. The measured fix in Lab 8 was a new table. Here the same fix was applied to a live table under load because CockroachDB can change a primary key online. What it cannot do is change what the application *reads*: an ordered scan by `id` now fans out across 16 buckets (Lab 8 Part D step 5). Check for one before you do this in production.

```
scripts/incident stop 1
```

Part B: Incident 2 — "The home page is slow, the database is idle" (12 min)

```
scripts/incident start 2
```

The page: *p99 for /home is 6× normal since the marketing campaign started. Database CPU is flat. The web team says "it must be the database".*

- Steps 1–3** come back clean: healthy cluster, CPU low everywhere, no hot range worth the name (a few hundred QPS on one small range). Low CPU with high latency means the requests are *waiting*, not working.
- Step 4 — who is waiting on whom?** Console → SQL Activity → **Transactions**: sort by contention time. From SQL:

```

SELECT count(*) AS contention_events, max(contention_duration) AS longest_wait
FROM crdb_internal.transaction_contention_events
WHERE collection_ts > now() - INTERVAL '1 minute';

SELECT substring(metadata->>'query', 1, 70) AS stmt,
       (statistics->'statistics'->>'cnt')::INT AS executions,
       round((statistics->'statistics'->'svcLat'->>'mean')::FLOAT * 1000, 1) AS mean_ms
FROM crdb_internal.statement_statistics
WHERE metadata->>'query' LIKE '%page_hits%'
ORDER BY executions DESC LIMIT 3;

```

Hundreds of contention events a minute, all on one key, and a single-row `UPDATE` whose mean latency is several times what a single-row update costs. No 40001s — a single-statement transaction is retried server-side — so nobody saw an error. They only saw the queue.

3. Read the application:

```
cat /tmp/oncall/2/loop.sh
```

Sixteen workers, and every request does `UPDATE page_hits SET hits = hits + 1 WHERE page = '/home'`. This is Playbook #5's anti-pattern, in production.

4. **Fix — this one is a deploy.** The database cannot make sixteen writers to one row not serialise; the *application* has to stop asking it to. Create the sharded table, then change the statement the application runs:

```

CREATE TABLE oncall.page_hit_shards (
  page STRING,
  shard INT2,
  hits INT NOT NULL DEFAULT 0,
  PRIMARY KEY (page, shard)
);
INSERT INTO oncall.page_hit_shards (page, shard) SELECT '/home', g FROM
generate_series(0, 15) g;

```

Deploy the new version of the application — the same file with one line changed:

```

cat > /tmp/oncall/2/loop.sh <<'EOF'
# web tier: 16 workers; each request increments ONE of 16 shards, chosen by the app
for w in $(seq 1 16); do
  ( for i in $(seq 1 40); do
      echo "UPDATE oncall.page_hit_shards SET hits = hits + 1 WHERE page = '/home' AND
        shard = $((RANDOM % 16));"
    done | ./cockroach sql --url 'postgresql://root@crdb1:26257/oncall?sslmode=disable' >/
    dev/null 2>&1 ) &
done
wait
EOF

```

The shell picks the shard — the server sees a literal. (Lab 8 Part E showed why `shard = (random()*16)::INT` in SQL is wrong.) The running container picks the new file up on its next loop, within a few seconds.

5. **Verify** after ~30 s — re-run the two queries from step 2. Contention events drop to a handful; the new statement's mean latency is a fraction of the old one's. The read side:

```
SELECT sum(hits) FROM oncall.page_hit_shards WHERE page = '/home';
```

Debrief. Nothing in the database was broken, and no database setting fixes this. The on-call engineer's job was to prove *where* the time went (a wait on one key) so the right team fixed the right thing. Contention that never produces an error is the hardest kind to get anyone to believe.

```
scripts/incident stop 2
```

Part C: Incident 3 — "The replication report is red since the rack move" (12 min)

```
scripts/incident start 3
```

Give the breaker a minute; it also has to commission a node.

The page: *system.replication_constraint_stats* has shown a violation on *oncall.pins* since node 4 was rebuilt during last night's rack move. Ops confirms node 4 is up and in *us-east1*.

1. **Step 1.** Four live nodes. Nothing unavailable or under-replicated. So far the cluster agrees with ops.
2. **What is the constraint, and is it met?**

```
SHOW ZONE CONFIGURATION FROM TABLE oncall.pins;  
SELECT type, config, violating_ranges  
FROM system.replication_constraint_stats WHERE violating_ranges > 0;  
SELECT lease_holder, replicas FROM [SHOW RANGES FROM TABLE oncall.pins WITH DETAILS];
```

The table asks for one replica and the lease in *us-east1*. The report says one range violates that. The lease is on node 1, 2 or 3 — not on the node that is supposed to be in that region.

3. **What does the cluster think node 4 is?**

```
SELECT node_id, address, locality FROM crdb_internal.gossip_nodes ORDER BY node_id;
```

region=us-esat1. The node is up, healthy, in the cluster — and, as far as every zone configuration is concerned, in a region nobody asked for. This is the silent failure from Day 2: a typo in a locality string does not error; it just never matches.

4. **Find where the string came from.** Ops rebuilt the node from a runbook:

```
cat /tmp/oncall/3/start-node4.sh
```

5. **Fix — correct the runbook and run it.** The node's store is on a named volume, so this restarts the *same* node with its data, not a new one:

```
sed -i.bak 's/us-esat1/us-east1/' /tmp/oncall/3/start-node4.sh    # or open it in an editor
bash /tmp/oncall/3/start-node4.sh
```

6. **Verify.** The report refreshes about once a minute; the lease moves as soon as the allocator sees a node that satisfies the preference:

```
SELECT node_id, locality, is_live FROM crdb_internal.gossip_nodes ORDER BY node_id;
SELECT lease_holder, replicas FROM [SHOW RANGES FROM TABLE oncall.pins WITH DETAILS];
SELECT count(*) AS violating FROM system.replication_constraint_stats WHERE
    violating_ranges > 0;
```

Lease on node 4; violations 0.

Debrief. Two versions of this failure exist. `CONFIGURE ZONE` with a constraint no node satisfies is *refused* — friendly. A node whose locality later stops matching is *accepted* — the constraint just goes unmet, forever, with the node reporting healthy. `system.replication_constraint_stats` (and its siblings `replication_critical_localities` and `replication_stats`) are the only place this shows up. Alert on them.

```
scripts/incident stop 3
```

Part D: Incident 4 — "Disk is filling and the table is tiny" (14 min)

```
scripts/incident start 4
```

The page: *Storage used is climbing steadily on all three nodes. The only busy table is `oncall.sessions`, and it has 2,000 rows.*

1. **Step 5 — what is growing?**

```
SELECT count(*) AS rows, round(sum(length(blob)) / 1e6, 2) AS live_mb FROM
    oncall.sessions;
SELECT range_id, round(range_size_mb) AS size_mb
FROM [SHOW RANGES FROM TABLE oncall.sessions WITH DETAILS];
```

About a megabyte of live data in a range of tens of megabytes — and run it again in a minute: the range grows, the row count does not. That gap is MVCC history that garbage collection is not removing.

2. **Why isn't GC removing it?** The table's GC window is short:

```
SHOW ZONE CONFIGURATION FROM TABLE oncall.sessions; -- gc.ttlseconds = 60
```

so something is *protecting* older versions. Protected timestamps are how backups, changefeeds and imports pin history:

```
SELECT ts, decoded_meta, decoded_target FROM crdb_internal.kv_protected_ts_records;
```

One record, owned by a job. Look the job up:

```
SELECT job_id, status, running_status, created FROM [SHOW CHANGEFEED JOBS];
```

A **paused** changefeed. Every version of every row written since it paused is being kept for a consumer that is not consuming.

3. **Read the application** (`cat /tmp/oncall/4/loop.sh`): a session store rewriting 500-byte blobs continuously. Each rewrite is a new MVCC version; at this rate the pinned history grows by tens of megabytes a minute.
4. **Decide, then fix.** Two correct answers, depending on a question only the business can answer — *is anything going to consume this feed?*

- **No** (its sink is `null://` ; nobody is reading): cancel it. The protected timestamp is released immediately.

```
CANCEL JOBS (SELECT job_id FROM [SHOW CHANGEFEED JOBS] WHERE status = 'paused');
```

- **Yes:** resume it. It catches up, and its protected timestamp advances with its checkpoint — but by default that record is refreshed only every 10 minutes (`changefeed.protect_timestamp_interval`), so the garbage clears on that schedule, not instantly.

Use `CANCEL` here.

5. **Verify.** The record is gone at once; the storage layer notices within about two minutes (`kv.protectedts.poll_interval`), and the next GC pass on the range removes everything older than 60 s:

```
SELECT count(*) AS protected_records FROM crdb_internal.kv_protected_ts_records;
-- ask the GC queue to look at the range now rather than on its own schedule
-- (the function acts on this node's replica, hence the filter on node 1 – your gateway)
SELECT crdb_internal.kv_enqueue_replica(range_id, 'mvccGC', true)
FROM [SHOW RANGES FROM TABLE oncall.sessions WITH DETAILS] WHERE 1 = ANY(replicas);
SELECT round(range_size_mb) FROM [SHOW RANGES FROM TABLE oncall.sessions WITH DETAILS];
```

Repeat the last two statements a few times over the next three minutes. Before the fix the size only went up; after it, it falls back toward the live size each time GC runs and saw-tooths there while the application keeps writing. Console → **Storage** shows the same curve.

Debrief. A paused job is not free. The same protection that lets a changefeed resume without losing events lets it hold the entire cluster's history hostage. `SHOW JOBS` filtered to `paused` belongs on the daily checklist, and `kv_protected_ts_records` is how you find out *what* is pinning storage when the answer is not obvious.

```
scripts/incident stop 4
```

Optional — If Time Allows

Part E: Incident 5 — "Patch the cluster without anyone noticing" (15 min)

```
scripts/incident start 5
docker logs -f oncall-app-5      # one line per 20-second run: time, errors, ops, latencies.
                                Ctrl+C to stop following
```

Nothing is broken. An order service is running against the cluster, and a patch release (v23.2.6) has to go out. The procedure is a **rolling restart**: one node at a time, wait for it to rejoin, move on.

1. Restart node 2 on the new version:

```
scripts/crdb upgrade 2 v23.2.6
```

Watch the app's log: a few errors at most, throughput dips for a moment. Wait until `scripts/crdb status` shows all three live, then do node 3 the same way.

2. Now node 1. The app's log shows thousands of errors in that window. Read why:

```
cat /tmp/oncall/5/loop.sh
```

The service connects to `crdb1` — one address. Restarting *that* node is an outage for *that* application, however healthy the cluster is.

3. Fix — in the connection string. PostgreSQL drivers accept several hosts and fail over between them; so does every load balancer. Deploy the change:

```
sed -i.bak 's|root@crdb1:26257/kv|root@crdb1:26257,crdb2:26257,crdb3:26257/kv|' /tmp/
oncall/5/loop.sh
grep postgresql /tmp/oncall/5/loop.sh
```

Then restart node 1 again (back to `v23.2.5`, to prove the direction does not matter):

```
scripts/crdb upgrade 1 v23.2.5
```

Errors in that window: zero or single digits; latency spikes briefly and recovers.

4. Check the versions and the cluster version:

```
SELECT node_id, server_version FROM crdb_internal.gossip_nodes ORDER BY node_id;
SHOW CLUSTER SETTING version;
```

Both read 23.2: a *patch* upgrade changes binaries, not the cluster version. A *major* upgrade (23.2 → 24.1) has one more step — after every node runs the new binary the cluster version is finalised, automatically after some time or explicitly with `SET CLUSTER SETTING version = '24.1'`; until then the cluster can still roll back. `cluster.preserve_downgrade_option` holds it open on purpose.

Debrief. The database did its part in every restart: leases moved, quorum held, nothing was lost. Whether the *application* noticed was decided entirely by a string in its config. This is the VM/bare-metal version of Lab 16's old Kubernetes lesson: the platform restarts nodes safely; your connection string decides whether that is invisible.

```
scripts/incident stop 5
```

Cleanup

```
scripts/incident stop all
scripts/crdb down
```

Lab 16 Deliverables

Your incident log:

#	First signal you saw	Root cause	Fix (DB or deploy?)	How you verified	Time to diagnose
1					
2					
3					
4					
5					

□ Four incidents worked in the fixed order, each verified rather than assumed fixed □ One fix in DDL, one in a job, one in a runbook, one in application code — and the reasoning for each □ (Optional) A rolling restart with and without a multi-host connection string, with the error counts

Challenge Exercises

1. **Write the alerts.** For each incident, the Prometheus expression (Lab 9) that would have paged *before* a human noticed. Incident 3's is the interesting one.
2. **Incident 2, the other way.** Instead of sharding, make the web tier's statement `SELECT ... FOR UPDATE` first (Lab 5 Part D). Measure mean latency against the sharded version. Which would you ship, and why does the answer depend on the read side?
3. **Incident 4 without cancelling.** Resume the job and lower `changefeed.protect_timestamp_interval` to `30s`. How long until the range shrinks, and what does that setting cost on a cluster with hundreds of changefeeds?

Reference

Question	Query
Hot range?	Console Hot Ranges; <code>SHOW RANGES FROM TABLE t WITH DETAILS</code>
Who is waiting?	<code>crdb_internal.transaction_contention_events</code> , <code>crdb_internal.cluster_contention_events</code>
Statement cost	<code>crdb_internal.statement_statistics (svcLat, cnt, maxRetries)</code>
Placement unmet?	<code>system.replication_constraint_stats</code> , <code>system.replication_critical_localities</code>
What does a node advertise?	<code>crdb_internal.gossip_nodes (locality)</code>
What pins history?	<code>crdb_internal.kv_protected_ts_records</code> → <code>SHOW JOBS</code>
Force a GC pass	<code>crdb_internal.kv_enqueue_replica(range_id, 'mvccGC', true)</code>
Rolling restart	<code>scripts/crdb upgrade N <version> (one node, docker compose up -d --no-deps)</code>